

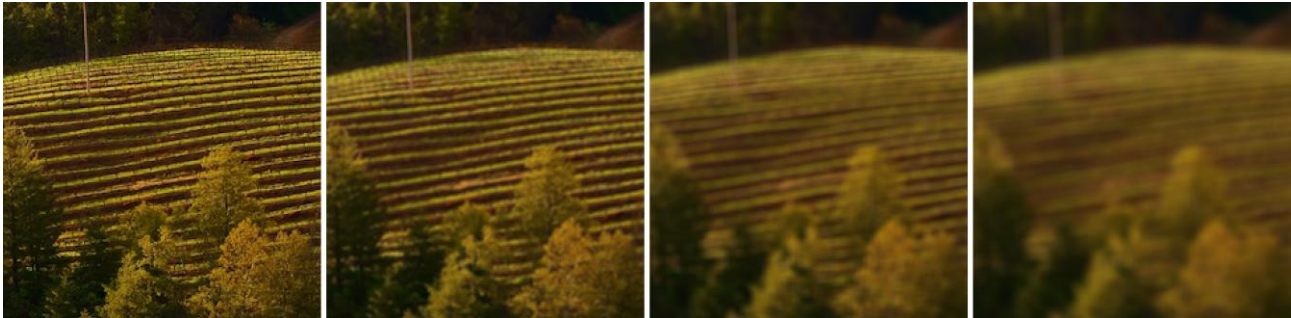
Homework 1 — Image Processing

Computer Vision (2026) · Dept. of Applied Artificial Intelligence, SeoulTech

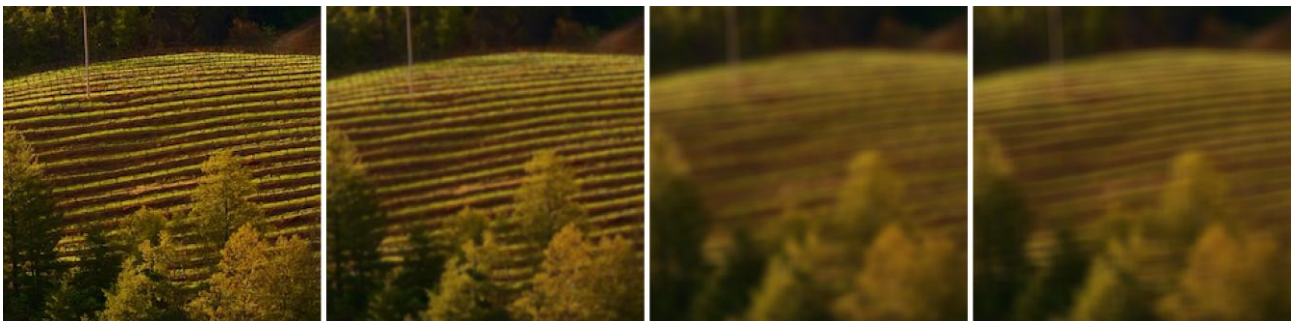
Student ID	22101355	Name	Gyuwan Choi
------------	----------	------	-------------

1. Results and observations

Problem 1. Linear filtering and smoothing



[figure: p1_ksize_sweep.png — original, ksize 3, 7, 11 at sigma = 4]



[figure: p1_sigma_sweep.png — original, sigma 1, 4, 9 at ksize = 11]



[figure: p1_pad_corner.png]

1-d-1 With sigma fixed at 4.0, how does the kernel size change the result? The script prints the fraction of the Gaussian that an 11x11 window actually holds — use that number to explain what goes wrong when the kernel is too small for the sigma.

Answer: With sigma fixed at 4 the blur gets stronger as the kernel gets bigger. At ksize 3 the vine rows look almost the same as the original, at 7 they are softer, and at 11 I can barely see them.

I expected sigma alone to decide how much the image is smoothed. But the script prints that an 11x11 window only holds 69.2% of a sigma = 4 Gaussian. I ran the same calculation for the other sizes and got 38.4% for 7x7 and 8.6% for 3x3. So the window cuts off a large part of the Gaussian (most of it for 7x7 and 3x3), and since the kernel is normalized to sum to 1 afterwards, the filter that is actually applied is not the sigma = 4 Gaussian anymore.

The 3x3 kernel is the worst case. Its center weight is 0.116, which is almost $\frac{1}{9}$, so it is basically a 3x3 box filter. I also computed the standard deviation of each truncated kernel: 0.81 px for 3x3, 1.91 px for 7x7 and 2.79 px for 11x11, all well below 4. So when the kernel is too small for the sigma, the image is smoothed much less than sigma says, and the kernel is so flat that it acts like a box filter rather than a Gaussian. To really get sigma = 4 the kernel would need about $2[3\sigma] +$

1 = 25 taps.

1-d-2 With the kernel size fixed at 11, how does sigma change the result? State what sigma controls and what the kernel size controls, in one sentence each.

Answer: At ksize 11, sigma = 1 only removes the finest texture and sigma = 4 blurs the vine rows a lot, but sigma = 9 looks almost the same as sigma = 4. This is the same truncation problem as above. The 11x11 window holds 100% of a sigma = 1 Gaussian but only 21.1% of a sigma = 9 one, so the sigma = 9 kernel is nearly flat and is close to an 11x11 box filter (its standard deviation is 3.09 px, a real 11x11 box has 3.16 px, and the sigma = 4 kernel has 2.79 px).

Sigma controls how quickly the weights decrease away from the center, which sets how strongly the image is smoothed.

The kernel size controls how much of that Gaussian is actually kept, so it limits how far the smoothing can reach.

So the two have to be chosen together. As a rule I would set the kernel size to about $2\lceil 3\sigma \rceil + 1$, roughly $6\sigma + 1$.

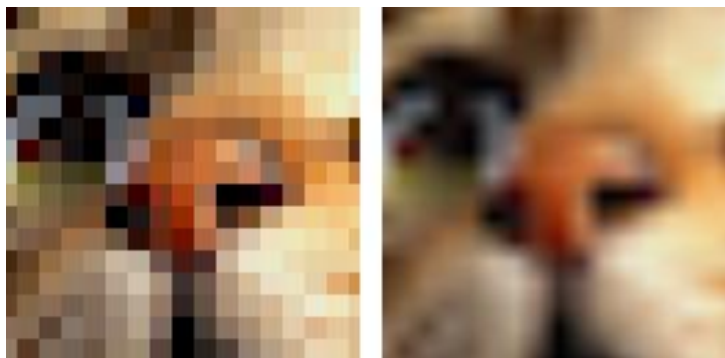
1-e Compare the two padding modes at the border, using both the corner figure and the printed border error. Why does zero padding darken the border? Which mode would you use for a smoothing filter, and why?

Answer: In the corner figure the zero-padded result has a dark band along the top and left edges and it is darkest right at the corner. The replicate result looks the same as the original. The band is about 12 px wide, which is half of the 25-tap kernel. The printed error says the same thing: over the top 12 rows the mean absolute difference from the original is 53.199 for zero padding and only 0.596 for replicate.

Zero padding darkens the border because the part of the kernel that hangs outside the image is multiplied by 0, while the kernel weights still sum to 1. Near the edge the output is therefore an average of real pixels and black pixels. At the corner only about a quarter of the kernel is inside the image, so that is where it gets darkest.

For a smoothing filter I would use replicate padding. It extends the image with its own border values, so a flat region stays flat up to the edge and the brightness does not change. A smoothing filter is an average, so it should not make the image darker.

Problem 2. Upsampling



[figure: p2_upsample_crop.png — nearest (left) and bilinear (right)]

2-c Describe the difference between the two crops. Name the artefact each method produces and say which you would choose for an 8x enlargement, and for what purpose.

Answer: Nearest neighbor sets each output pixel to the nearest source pixel, so every source pixel becomes an 8x8 square. The crop looks blocky, and curved edges like the eye outline and the lines around the nose turn into staircases. The artifact is blockiness (jagged edges). The good side is that no new values are created.

Bilinear interpolation uses the 4 neighbor pixels and takes a weighted sum based on the area of the opposite rectangle. The result is smoother than nearest neighbor and there are no blocks, but the artifact is blur: the edges look soft. The two outputs differ by 2.45 gray levels on average and by up to 126 at sharp edges.

If the 8x image is for viewing, I would choose bilinear because the blur bothers me less than the blocks. If I need to look at the actual pixel values, or the image is a label map or a mask where in-between values make no sense, I would choose nearest.

2-d Predict what changes if the source coordinate is $u = d \cdot \text{scale}$ instead of $u = (d + 0.5) \cdot \text{scale} - 0.5$. Then try it and report what you observed. One paragraph.

Answer: Prediction: the value of a pixel belongs to the center of its cell, and $u = (d + 0.5) \cdot \text{scale} - 0.5$ is what makes the centers of the two grids line up, so without the 0.5 terms I expected the whole image to be shifted. With $\text{scale} = \frac{1}{8}$ the difference between the two formulas is $0.5 - 0.5 \cdot \text{scale} = 0.4375$ source pixels, so every output pixel samples a bit further down and to the right, and the picture should move up and left by 3.5 output pixels. Observation: I changed source_coordinates to $d \cdot \text{scale}$ and ran both upsamplers on cat.png again. The nearest result matches the correct one exactly if I shift it by 4 px, and the bilinear result matches best at 3 px (the real shift is 3.5 px, which is not an integer). Without shifting, the mean difference from the correct output is 5.26 gray levels for nearest and 3.63 for bilinear. In the nearest result the first source row covers only 4 output rows and the last one covers 12, instead of 8 each. The sharpness looks the same, so nothing is blurred; the image is just misaligned by about half a source pixel, which would be a problem when comparing it with or overlaying it on another image.

Problem 3. Downsampling and aliasing



[figure: p3_alias_crop.png — no prefilter (left) and prefiltered (right)]

sigma	ksize	High-freq energy	Δ vs. naive
none	–	569.442	0.000
0.5	5	447.933	1.269
1.0	7	306.605	3.281
2.0	13	201.363	4.650

[figure: the printed sigma table]

3-c Point to a region of the naive result that shows structure which is not in the original image. Where did that structure come from? Use the term aliasing correctly.

Answer: In the naive result (left) the vineyard on the hill has wide curved bands, like rings, a few pixels apart. They are not in the original. The real vine rows are thin and almost parallel, and they run in a different direction from the bands. The prefiltered result on the right does not have them.

I checked the intensity profile in this region and the rows repeat about every 8 to 13 pixels in the original. Naive downsampling keeps only every 8th pixel, and a grid like that can only represent patterns with a period longer than 16 pixels (its Nyquist limit). The rows are finer than that, so they are not just lost. They show up as a different, slower pattern. For example a 12 px period sampled every 8 px looks like a 24 px period.

This is aliasing: the sampling is too coarse for the detail in the image, so the sampling distorts the original signal and the fine pattern appears as a coarse pattern that does not exist in the scene. The noisy, speckled look of the trees at the bottom of the naive crop is the same effect.

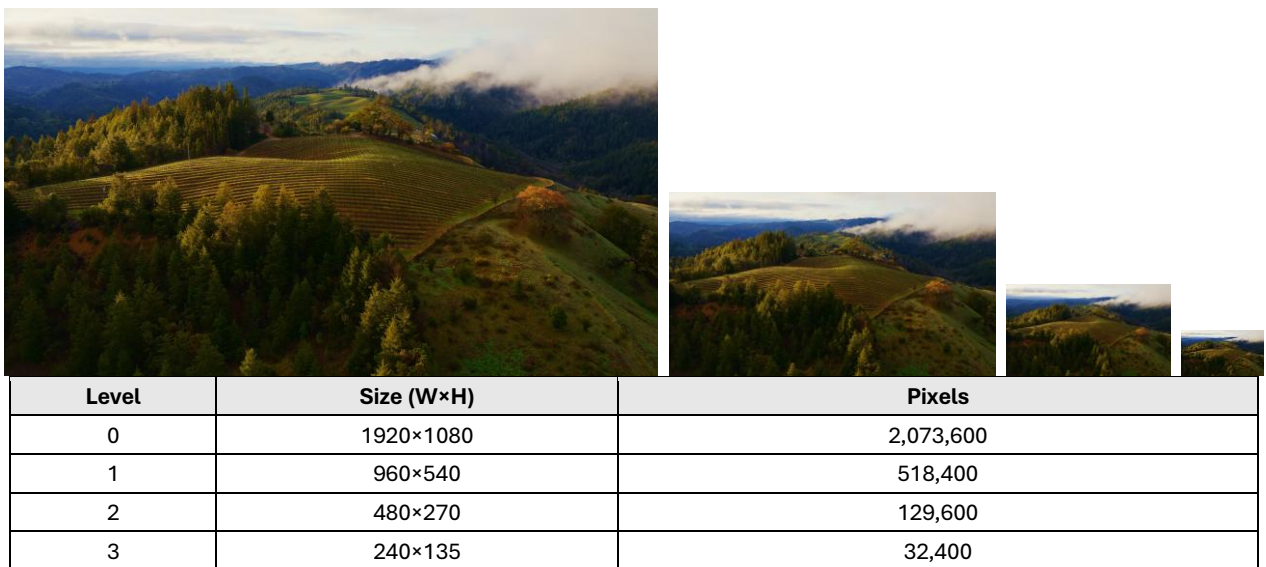
3-d As sigma grows, what happens to the aliasing and what happens to the detail? Which sigma would you choose for a reduction to one eighth, and on what grounds?

Answer: As sigma grows the aliasing goes down. In the sigma table the high-frequency energy drops from 569.4 without a prefilter to 447.9, 306.6 and 201.4 for sigma = 0.5, 1 and 2. In the images, sigma = 0.5 hardly changes the false bands, sigma = 1 makes them weaker but they are still there, and at sigma = 2 they are mostly gone.

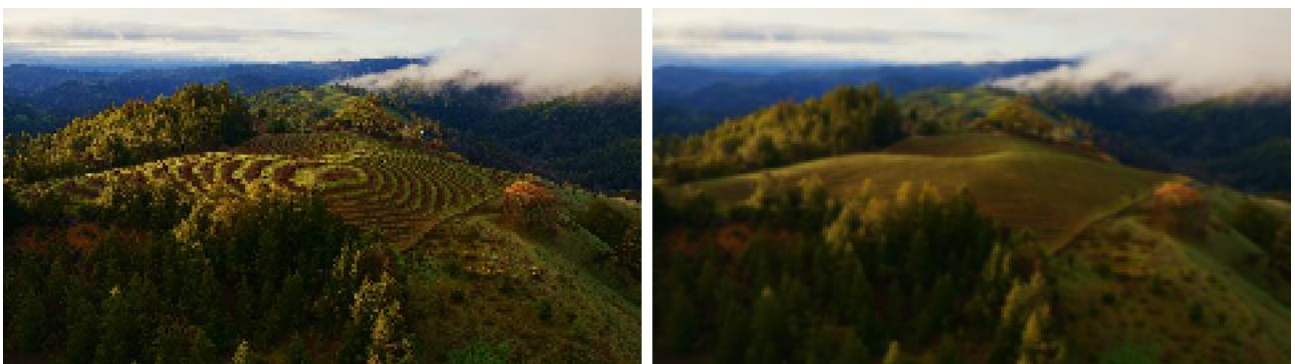
The detail also goes down. The mean difference from the naive image increases from 1.27 to 3.28 to 4.65, and the tree texture and the ridge line get softer. Some of the energy that was removed was real detail and not aliasing, so I don't think the energy column alone is enough to pick sigma.

For a reduction to one eighth I would choose sigma = 2 out of the values I tested, because it is the only one that really removes the false bands. I think a false pattern is worse than some blur, since blur only loses information and aliasing adds something wrong. This is why the image has to be smoothed before it is sampled. The larger the reduction, the more smoothing is needed, so sigma = 0.5 or 1 is too small for a factor of 8. A slightly bigger sigma, around 3, would probably remove the faint rows that are left, with more blur.

Problem 4. Gaussian pyramid



[figure: p4_level_0.png ... p4_level_3.png, with the size table]



[figure: p4_vs_naive.png — naive (left) and coarsest pyramid level (right)]

4-c Both results keep one pixel in sixty-four. Which looks better, and why? Answer in terms of what each method did to the frequencies the coarse grid cannot represent.

Answer: The pyramid result (right) looks better. The vineyard is a smooth slope with faint rows and the trees are soft but look natural. The naive result (left) has the strong curved bands on the vineyard and speckle in the trees, the same aliasing as in Problem 3.

The reason is what happens to the high frequencies that the small grid cannot represent. Naive downsampling just deletes pixels, so those frequencies are still in the signal when it is sampled and they come back as false low-frequency

patterns. The pyramid filters and then subsamples at every level, so before each step the detail that the next level cannot hold has mostly been smoothed away and cannot alias. The pyramid does lose detail, but only detail that the coarse grid could not show correctly anyway. The pyramid levels show the same thing: the details get smoothed out more at each level and mostly the large uniform regions remain.

Numerically the two results differ by 7.21 gray levels on average (max 158). Part of this is not aliasing but a 4 px offset, because the pyramid takes `array[:, ::2, ::2]` at each level while the naive sampler uses the center-aligned coordinates from Problem 2.

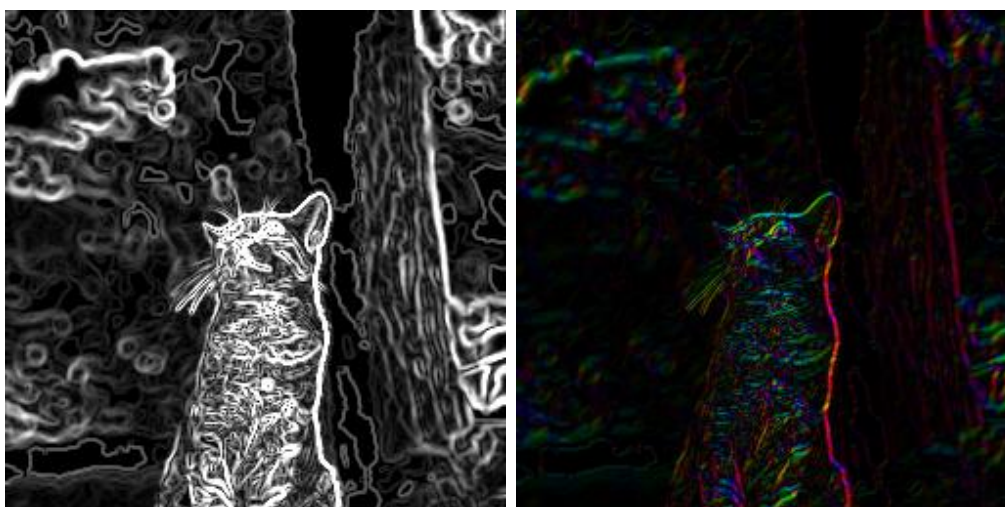
4-d The pyramid smooths once per level with $\sigma = 1$. How does the total smoothing compare with a single Gaussian applied to the original image? State the relation you used.

Answer: Each level smooths with $\sigma = 1$, but measured in its own pixels. The first smoothing is $\sigma = 1$ in original pixels. The second is done on the half-size image, so it corresponds to $\sigma = 2$ in original pixels, and the third is done at quarter size, so it corresponds to $\sigma = 4$.

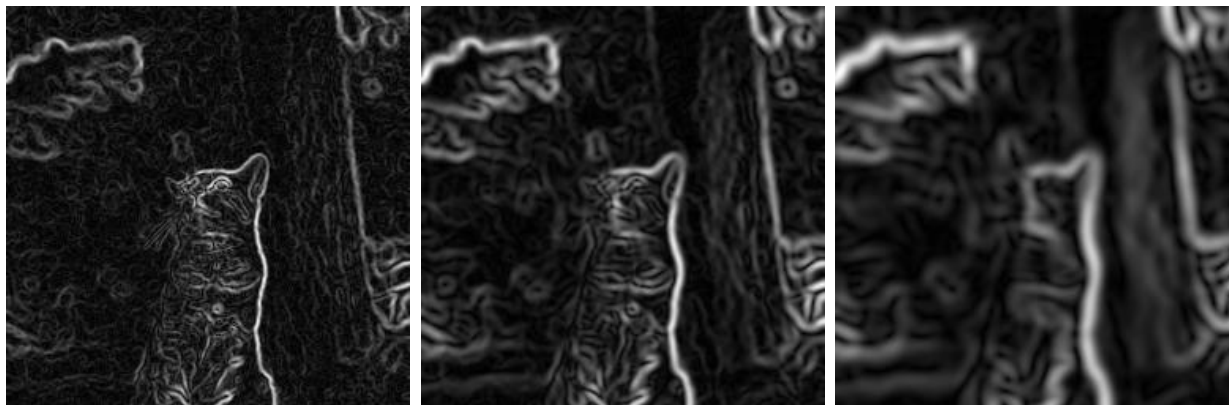
The relation I used is that convolving two Gaussians gives another Gaussian and the variances add: $G(\sigma_1) * G(\sigma_2) = G(\sqrt{\sigma_1^2 + \sigma_2^2})$. So the total smoothing for level 3 is $\sigma_{\text{eff}} = \sqrt{1^2 + 2^2 + 4^2} = \sqrt{21} \approx 4.58$ original pixels, followed by taking every 8th pixel. In general, for L levels, $\sigma_{\text{eff}}^2 = \sum_{l=0}^{L-1} (\sigma \cdot 2^l)^2$.

To check this I smoothed the original once with $\sigma = 4.58$ (ksize 29) and kept every 8th pixel. It differs from level 3 by only 0.07 gray levels on average, so the two are practically the same. The pyramid is much cheaper though, because the later 5x5 filters run on images with 4 times fewer pixels each time, instead of one 29x29 filter on the full image.

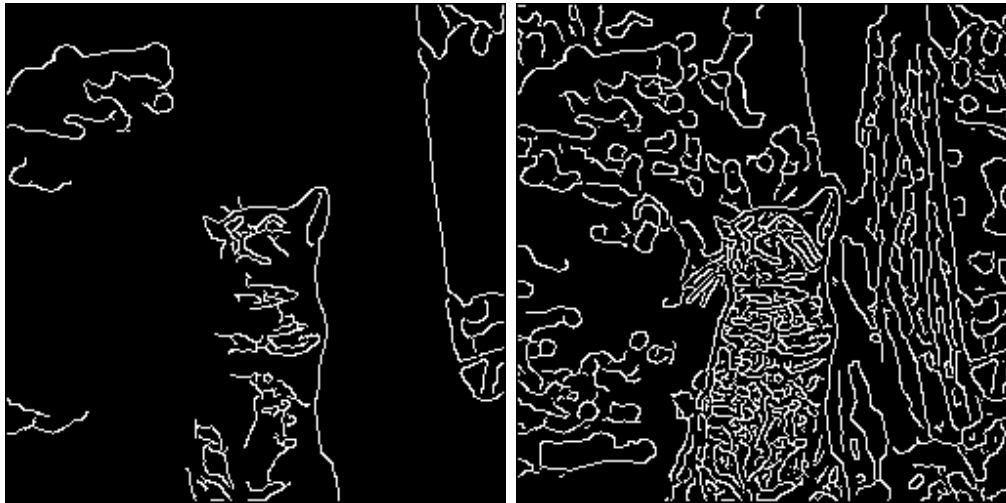
Problem 5. Edge detection



[figure: p5_sobel_magnitude.png and p5_sobel_orientation.png]



[figure: p5_dog_noisy_s1.png, p5_dog_noisy_s2.png, p5_dog_noisy_s4.png]



[figure: p5_canny_80_200.png and p5_canny_20_50.png]

5-a In the orientation image, what do you see along a long straight edge, and what do you see in a textured region? Explain the difference.

Answer: Along a long straight edge, for example the right outline of the cat or the sides of the tree trunk, the orientation image has a single color along the whole edge, and it is bright because the magnitude is large there. The gradient direction is the direction of the greatest rate of increase, which is perpendicular to the edge, and on a straight edge this direction is the same at every pixel.

In textured regions like the striped fur and the leaves in the background, the color changes from pixel to pixel and looks like colorful noise. It is also darker and uneven. Texture consists of many small intensity changes in all directions (single hairs, leaves), so the gradients of neighboring pixels point in different directions and there is no one dominant direction.

5-b On the noisy image, compare $\sigma = 1, 2$ and 4 . State the trade-off between suppressing noise and locating the edge accurately, and support it with the printed table.

Answer: A larger σ suppresses the noise better. In the printed table the noisy/clean ratio of the mean gradient magnitude goes from 1.206 at $\sigma = 1$ to 1.029 at $\sigma = 2$ and 1.004 at $\sigma = 4$, so the noise adds about 21% to the response at $\sigma = 1$ and almost nothing at $\sigma = 4$. The DoG images show the same: $\sigma = 1$ is full of speckle, $\sigma = 2$ is much cleaner, and $\sigma = 4$ has almost no noise. Differentiation is very sensitive to noise, so the image has to be smoothed first, and the DoG kernel does the smoothing and the derivative in one filter.

The price is that the edges are located less accurately. With a larger σ the response is spread over a wider area, so the peak response on the clean image drops from 72.0 to 37.7 to 20.9 and the edges become thick bands. The kernel also grows from 7 to 13 to 25. At $\sigma = 4$ the outline of the cat is several pixels wide, close edges like the whiskers and the fur stripes merge together, and small structures disappear.

So a small σ gives sharp, well-located edges but also responds to noise, and a large σ is robust to noise but the edges are wide and their position is less certain. In other words, a small σ suits fine features and a large σ suits large-scale edges. For this image (noise $\sigma = 15$) I think $\sigma = 2$ is a reasonable choice.

5-d Compare the two threshold settings. Which gives broken contours and which gives false edges? What does hysteresis achieve that a single threshold cannot?

Answer: With (low, high) = (80, 200) only 2434 pixels (3.7% of the image) are kept. The main contours are found but they are broken: there are gaps in the cat outline and the trunk edges, and most of the cat's body is missing. With (20, 50) there are 9655 edge pixels (14.7%). The contours are connected, but there are many false edges from the fur and the background leaves. So the high setting gives broken contours and the low setting gives false edges.

With a single threshold I would have to choose between these two problems: a high threshold removes true edges and a low one keeps false edges. Hysteresis uses two thresholds. A pixel above the high threshold is a strong edge. A pixel between the two thresholds is a weak edge, and it is kept only if it connects to a strong edge. Pixels below the low threshold are not edges. This way a real contour can continue through its weaker parts, while weak responses from noise or texture that are not connected to anything strong are removed.

2. Self-check output

Paste the complete output of python check.py below.

```
self-check on data\cat.png (256x256)

Problem 1. Linear filtering and smoothing
[PASS] 1-a kernel matches OpenCV          max err < 1e-9, sum = 1
[PASS] 1-b padding matches numpy          max err=0.00e+00
[PASS] 1-c filtering matches OpenCV       max err=0.00e+00 (2 padding modes)
[PASS] 1-c smoothing matches GaussianBlur max err=3.41e-13

Problem 2. Upsampling
[PASS] 2-a nearest matches OpenCV         mismatched pixels=0.0000%
[PASS] 2-b bilinear matches OpenCV        max err=0.742 (uint8 rounding)

Problem 3. Downsampling and aliasing
[PASS] 3-a naive output shape             (32, 32, 3)
[PASS] 3-b prefilter removes high freq    edge energy 3827.4 -> 2423.5

Problem 4. Gaussian pyramid
[PASS] 4-a level count and sizes          [(256, 256), (128, 128), (64, 64), (32, 32)]
[PASS] 4-a level 0 is the input           max err=0.00e+00

Problem 5. Edge detection
[PASS] 5-a Sobel matches OpenCV           max err=0.00e+00, magnitude err=0.00e+00
[PASS] 5-b DoG kernels are derivatives    |sum|=6.91e-18, sign ok=True, antisymmetric=True
[PASS] 5-c NMS thins without adding       kept 26.1% of pixels, values unchanged=True
[PASS] 5-c Canny matches OpenCV           worst F1=0.937 over 2 settings

14 passed, 0 failed, 0 skipped out of 14 checks
Paste this output into section 2 of your report.
```

Note on 5-b (sign of the kernel). The docstring gives $D_x = -\frac{t-c}{\sigma^2} G$, which is the derivative written for convolution. But filtering() does correlation, so with that kernel the response comes out negative where the intensity increases to the right. That would contradict the sign convention in the docstring ("as in sobel_kernels()") and the self-check would fail. So I used the flipped kernel $D_x = +\frac{t-c}{\sigma^2} G$ (and the same for D_y), which is the same derivative of Gaussian written for correlation.